



ECOLE
POLYTECHNIQUE
DE BRUXELLES

2025-2026

PROJ-H415

Machine Learning Project for Sustainability

Authors :

Laurent Béguelin

Professor :

Domenico Spina

2026



1 Introduction

This report presents two machine learning (ML) classification models built with the CatBoost gradient-boosting library to predict the annual energy consumption of buildings in the city of Chicago. The first model targets **electricity consumption** (measured in kilowatt-hours, KWH) and the second targets **gas consumption** (measured in Therms).

Rather than predicting exact consumption values, the models classify each building into one of five consumption ranges (classes), making this a *multi-class classification* problem. The dataset used is the publicly available Chicago Energy Benchmarking dataset for the year 2010.

2 Dataset Description

2.1 Source and Structure

The data are provided in two CSV files:

- `Project_Data_Electricity.csv` — yearly electricity consumption
- `Project_Data_Gas.csv` — yearly gas consumption

Each file contains ten columns: nine **feature** columns describing the building and one **label** column (TOTAL_KWH or TOTAL_THERMS).

Column	Description
COMMUNITY AREA NAME	Neighbourhood in Chicago (categorical)
BUILDING TYPE	Residential / Commercial / Industrial (categorical)
TOTAL POPULATION	Population in the community area
TOTAL UNITS	Number of units in the building
AVERAGE STORIES	Average number of stories
AVERAGE BUILDING AGE	Average age of buildings in years
AVERAGE HOUSESIZE	Average household size
OCCUPIED UNITS PERCENTAGE	Fraction of occupied units

Table 1: Feature columns common to both datasets

3 Data Cleaning and Preprocessing

3.1 Target Classes

first we made class for each target they are in the following table : Instead of regressing to an exact value, each observation is assigned to a consumption class.

Class	Electricity (KWH)	Gas (Therms)
1	$\leq 30,000$	$\leq 5,000$
2	$30,000 < x \leq 60,000$	$5,000 < x \leq 10,000$
3	$60,000 < x \leq 90,000$	$10,000 < x \leq 15,000$
4	$90,000 < x \leq 120,000$	$15,000 < x \leq 20,000$
5	$> 120,000$	$> 20,000$

Table 2: Class definitions for electricity and gas consumption

3.2 Handling Missing Values and One-hot Encoding

Several rows lacked a target value. All rows containing any missing data were dropped, and rows with TOTAL KWH but 0 Units were also drop for simplicity, and finally index row were reset for clarity. One hot encoding was also applied to change categorical features into numerical features without a specific order of importance. an example is in the table below :

Residential	Commercial	Industrial
1	0	0
0	0	1
0	1	0
0	1	0
1	0	0

Table 3: One-hot encoding example for BUILDING TYPE(5 rows only)

After encoding, the feature set expanded to 87 columns, which were then fed into the model.

4 Model Development

4.1 Train / Validation / Test Split

The dataset was divided into 70% for training, 15% for validation, and 15% for testing. Alternative splits, such as 80/10/10 (training/validation/testing), could also be considered. However, the 70/15/15 partition was chosen arbitrarily for this work.

To preserve the class distribution across the different subsets, a stratified splitting strategy was applied using Python’s `stratify` parameter. This was necessary due to the imbalance in the dataset, where certain classes (notably classes 1 and 5) contained significantly more samples than others.

The class distributions for the full dataset are reported in Table 4.

Class	TOTAL THERMS	TOTAL KWH
1	20273	21517
2	13182	12890
3	9377	8754
4	7620	7097
5	12036	13476

Table 4: Class distribution in the dataset.

4.2 CatBoost Classifier

The gradient boosting algorithm underlying CatBoost rests on two foundational ideas. The first is the boosting theorem, introduced by Robert E. Schapire, which establishes that an ensemble of many weak learners can collectively form a single strong learner. The second is gradient boosting, formalised by Jerome Friedman [Fri01], which frames the ensemble construction as a gradient descent problem: at each iteration, a new tree is fitted to the residual errors of the current ensemble, progressively reducing the loss in the direction of the negative gradient — a principle shared with the optimisation of neural networks.

At inference time, the final leaves of the trees produce a vector of raw scores, one per class, which are converted into class probabilities via the softmax function:

$$P(y = k | \mathbf{z}) = \frac{e^{z_k}}{\sum_{j=1}^5 e^{z_j}}, \quad k = 1, \dots, 5 \quad (1)$$

where $\mathbf{z} = (z_1, \dots, z_K)$ is the vector of raw scores output by the ensemble.

The predicted class is then taken as the argument maximising this distribution.

4.3 Hyperparameter initial Choices

Before any hyperparameter tuning, the CatBoost classifier was initialised with the following configuration. Each parameter was chosen to provide a reasonable baseline for multiclass classification on an imbalanced dataset. These values served as the baseline configuration prior to tuning. Table 5 summarises the key parameters and their rationale.

Parameter	Value	Rationale
loss_function	MultiClass	Softmax cross-entropy for > 2 classes
eval_metric	Accuracy	Intuitive overall correctness measure
iterations	500	Conservative upper bound before tuning
learning_rate	0.1	Default; balances speed vs. accuracy
depth	6	Default tree depth; avoids overfitting
auto_class_weights	Balanced	Corrects for class imbalance
random_seed	42	Ensures reproducibility
verbose	100	Logs progress every 100 iterations

Table 5: CatBoost baseline hyperparameters (before tuning)

5 Results

Table 6 reports the test-set performance of both models before and after Optuna hyperparameter tuning, across four evaluation metrics.

Model	Phase	Accuracy (%)	F1 Score (%)	Precision (%)	Recall (%)
Electricity	Before tuning	64.20	65.13	67.05	64.20
	After tuning	64.33	65.23	67.14	64.33
Gas	Before tuning	80.57	80.68	81.97	80.57
	After tuning	80.66	80.75	82.04	80.66

Table 6: Test-set performance of the CatBoost classifiers before and after hyperparameter tuning

The four metrics reported are defined as follows. **Accuracy** measures the overall proportion of

correctly classified observations. **Precision** quantifies, among all observations predicted to belong to a given class, the fraction that truly belong to it. **Recall** measures, among all observations that truly belong to a given class, the fraction that were correctly identified. Finally, the **F1 Score** is the harmonic mean of precision and recall, providing a single balanced measure that accounts for both false positives and false negatives. All metrics are computed in their macro-averaged form, treating each consumption class equally regardless of its frequency in the dataset.

Both models benefit modestly from hyperparameter tuning, with consistent improvements across all four metrics. The gas model substantially outperforms the electricity model, achieving an accuracy above 80% compared to approximately 64% for electricity. This performance gap likely reflects greater variability in electricity consumption patterns across building types, resulting in less separable class boundaries and a more challenging classification problem.

5.1 Confusion Matrix

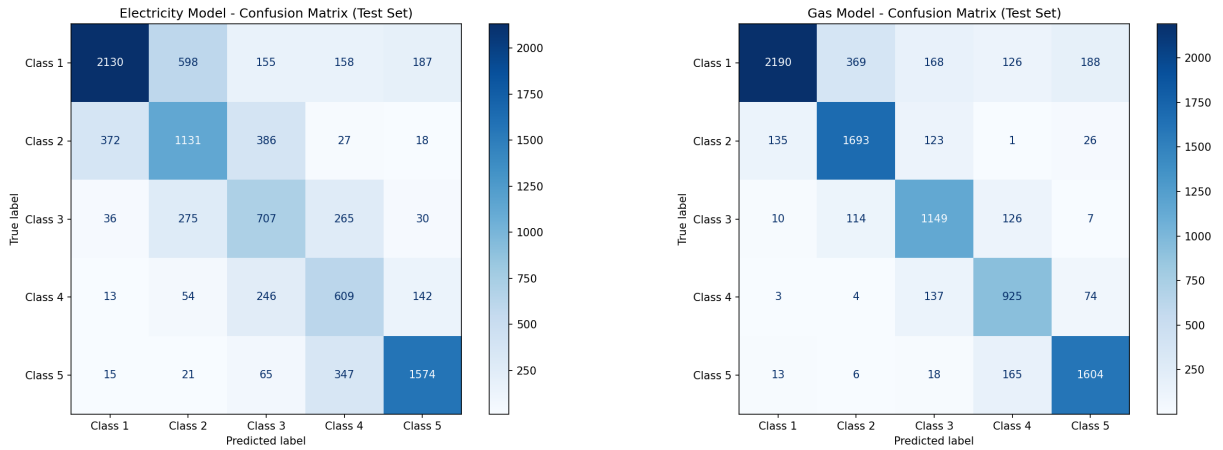


Figure 1: Confusion matrix for the electricity consumption(left) and gas consumption(right) model (test set) Rows represent true classes; columns represent predicted classes.

the confusion matrix are illustrated above 3. The diagonal elements represent correctly classified observations. A clear asymmetry is visible: classes at the extremes (far left and far right) are rarely misclassified, as they lie well away from any decision boundary. Middle classes (2–4), however, show higher confusion rates, as data points near the discretisation thresholds can plausibly belong to either neighbouring class. The gas model exhibits the same pattern: boundary classes are classified with high confidence, while middle classes remain the primary source of misclassification, consistent with the inherent ambiguity of fixed discretisation thresholds.

6 Discussion

6.1 Model Performance

The gas model substantially outperforms the electricity model, which can be attributed to two main factors. First, the consumption label corresponds to January — a winter month — during which gas usage for heating is high and predictable, making the total therms figure a strong and consistent signal. Second, electricity consumption is largely driven by occupant behaviour, yet the dataset primarily contains physical building characteristics. This mismatch between available features and the behavioural nature of electricity use likely accounts for the lower predictive accuracy of that model.

6.2 Carbon Footprint

Both models were evaluated for their environmental impact using the CodeCarbon Python library. The estimated CO₂ emissions were 3.96×10^{-5} kg for the electricity model and 4.02×10^{-5} kg for the gas model per training run — roughly equivalent to a single breath of air, confirming that the computational cost of these models is negligible from an environmental standpoint.

6.3 Class Imbalance

As visible in the confusion matrices, prediction accuracy is not uniform across classes. Extreme classes (1 and 5) are classified more reliably, as their data points lie far from any decision boundary. In contrast, middle classes (3 and 4) exhibit higher confusion rates, since observations near the discretisation thresholds can plausibly belong to either neighbouring class.

7 Additional Work

Feature Importance

The contribution of each feature to the model predictions was assessed and plotted in decreasing order of importance.

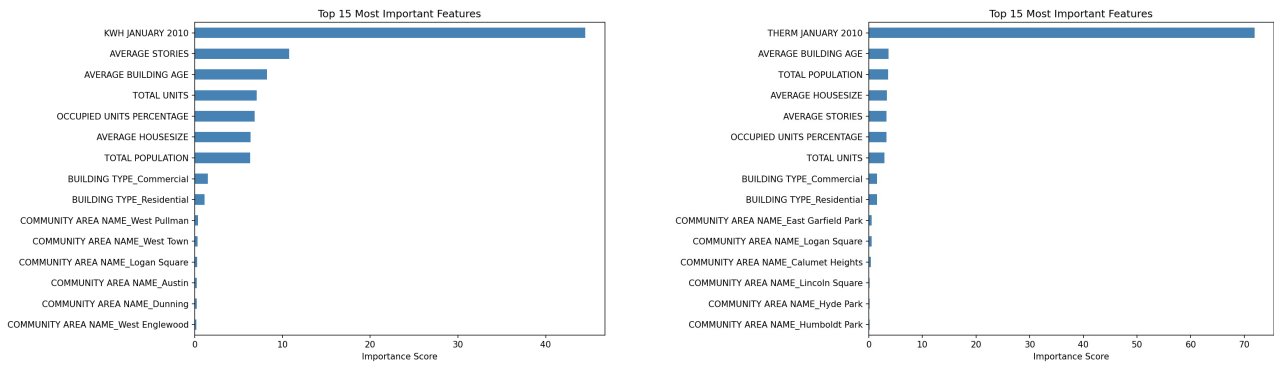


Figure 2: Features importance for electricity(left)and gas model(right)

In both models, the dominant contributors are the THERM and KWH variables. To explore whether richer representations could improve performance, a set of engineered features was constructed and evaluated.

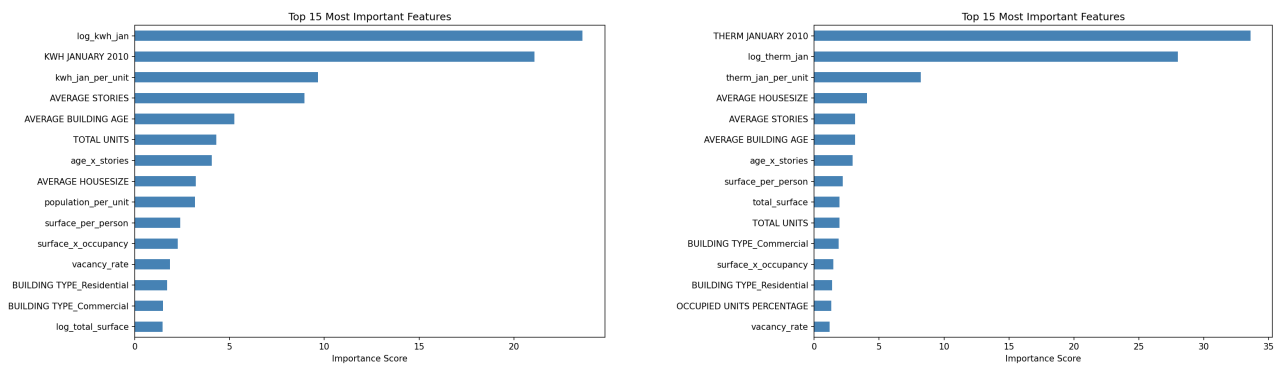


Figure 3: Features importance for electricity(left)and gas model(right)Using features engineering.

The engineered features yielded a marginal decrease in accuracy, suggesting that this approach was not beneficial in this context. That said, feature engineering remains a powerful technique in other domains — in finance, for instance, derived quantities such as volatility or momentum are routinely used to improve model performance.

Hyperparametrization

Another commonly used method is the optimization of the parameters; for that I used Optuna [Aki+19], which integrates well with Python and uses Bayesian optimisation guided by a target evaluation metric — in my case, accuracy. The results were improved by approximately 0.01%, yielding 64.33% accuracy compared to the previous 64.20% for the electricity model, and 80.66% compared to 80.57% for the gas model.

Table ?? lists only the parameters that were either updated or newly introduced by Optuna relative to the baseline configuration.

Parameter	Baseline	Tuned Value
iterations	500	1193
learning_rate	0.1	0.0954
l2_leaf_reg	—	2.158
colsample_bylevel	—	0.540
min_data_in_leaf	—	45

Table 7: CatBoost hyperparameters updated after Optuna tuning

The three newly introduced parameters (`l2_leaf_reg`, `colsample_bylevel`, `min_data_in_leaf`) act as regularisation terms that Optuna identified as beneficial: L2 leaf regularisation penalises large leaf values, column subsampling reduces overfitting by restricting the feature subset considered at each split, and the minimum leaf size prevents the model from fitting overly specific patterns in small leaf nodes.

Limitations and Future Improvements

The dataset covers only 2010, limiting temporal generalisability; incorporating data from additional years or with richer features would likely yield meaningful gains. Furthermore, the fixed class boundaries used for discretisation could be revisited, as alternative thresholds may produce better-balanced classes and higher accuracy. Finally, replacing the fixed 70/15/15 split with k -fold cross-validation would provide a more robust estimate of model performance.

8 Conclusion

Two CatBoost classifiers were trained to predict the annual electricity and gas consumption class of buildings in Chicago. After preprocessing — removing missing values, correcting numeric formatting, one-hot encoding categorical features, and discretising target variables into class labels — the models achieved test-set accuracies of **64.33%** (electricity) and **80.66%** (gas).

This project served as a valuable introduction to the machine learning pipeline, from data cleaning and feature engineering through to model selection and hyperparameter optimisation. It laid a meaningful foundation for the study of more complex architectures such as neural networks. I would like to thank Prof. Domenico Spina for proposing this project and for his guidance throughout.

Python code

the full python code for this project can be found on my github at this address :

<https://github.com/greenpepperlb/ML-project>

Bibliography

- [Aki+19] Akiba, Takuya et al. (2019). “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 2623–2631. DOI: 10 . 1145/3292500 . 3330701.
- [Fri01] Friedman, Jerome H. (Oct. 2001). “Greedy Function Approximation: A Gradient Boosting Machine”. In: *The Annals of Statistics* 29.5, pp. 1189–1232. DOI: 10 . 1214 / aos / 1013203451.